

GitHub Actions — YAML Syntax, Explained

Companion to the CI/CD theory notes — focused on why each piece of syntax exists

File location

Every workflow is just a YAML file placed in **.github/workflows/** at the root of your repo. GitHub scans that exact folder — nothing runs if the file is elsewhere. You can have multiple files here (e.g. tests.yml, deploy.yml), and each one is a completely independent workflow.

name

Purely cosmetic — it's the label shown at the top of the Actions tab so you can tell workflows apart at a glance. If you skip it, GitHub just uses the file path instead.

```
name: CI Pipeline
```

on — triggers

This is the "when does this run" part. Every workflow needs at least one trigger, or it never fires. You can trigger off a single event, a list of events, or a detailed config per event with filters.

```
on: push                # runs on every push, to any branch

on: [push, pull_request] # shorthand for multiple events

on:
  push:
    branches: [main]      # only when pushing to main
    paths: ['src/**']     # only if files under src/ changed
  pull_request:
    branches: [main]     # only PRs targeting main
  schedule:
    - cron: '0 6 * * *'  # cron: minute hour day month weekday
  workflow_dispatch:      # adds a manual "Run workflow" button
  workflow_call:          # lets other workflows call this one
```

Why the branch/path filters matter: without them, a workflow re-runs for every single push, even if it's just a README typo fix. Filters keep CI minutes from being wasted on irrelevant changes — relevant for something like your AQI predictor repo where you might only want the pipeline to run on a schedule, not on every doc update.

jobs

A workflow is made of one or more jobs. By default, GitHub runs all jobs in **parallel**, each on its own fresh virtual machine — they don't share filesystem or memory unless you explicitly pass data between them (via artifacts, or outputs).

```
jobs:
  test:
    name: Run unit tests      # job id — your own name, used to reference it
    runs-on: ubuntu-latest   # readable label shown in the UI
    timeout-minutes: 15      # which OS image the runner uses
    if: github.ref == 'refs/heads/main' # kill the job if it hangs this long
    # only run under this condition

  deploy:
```

```
needs: test                # waits for the "test" job to finish successfully first
runs-on: ubuntu-latest
```

needs is what turns parallel jobs into a pipeline. Without it, a deploy job could start running before tests even finish — needs forces sequencing, and if the job it depends on fails, this one is skipped automatically.

steps

Steps run in order, top to bottom, inside a job, and they share the same filesystem/workspace (unlike jobs, which don't). There are exactly two kinds of step:

- **uses:** runs someone else's packaged Action — reusable code, like a function call.
- **run:** executes a raw shell command directly on the runner.

```
steps:
  - name: Checkout code          # human-readable label for this step
    uses: actions/checkout@v4    # pulls your repo's code onto the runner

  - name: Print a message
    run: echo "hello"            # a single shell command

  - name: Multi-line shell script
    run: |                       # the pipe (|) lets you write multiple lines
      echo "line 1"
      echo "line 2"

  - name: Action that takes inputs
    uses: actions/setup-python@v5
    with:                         # "with" passes parameters into the action
      python-version: '3.11'
```

Why checkout has to come first: a runner starts as a completely blank VM. It doesn't know your repo exists until actions/checkout@v4 clones it in. Forgetting this step is the most common beginner mistake — every subsequent step that touches your files will fail.

env — environment variables

env can be set at three different levels, and the scope decides how widely it applies:

```
env:                          # workflow-level: available to every job
  NODE_ENV: production

jobs:
  build:
    env:                        # job-level: available to every step in this job
      DEBUG: true
    steps:
      - run: python app.py
        env:                    # step-level: only this one step can see it
          API_KEY: ${ secrets.MY_API_KEY }
```

Narrower scope (step-level) is generally safer for sensitive values like API keys, since it limits which parts of the workflow can accidentally leak or misuse them.

secrets

Secrets are encrypted values stored in your repo under **Settings** → **Secrets and Variables** → **Actions** — never committed to the YAML file itself. GitHub automatically masks their value in logs (shows ******* even if a step accidentally

prints it).

```
{{ secrets.SECRET_NAME }}
```

This is exactly how you'd pass something like a Hopsworks API key or Open-Meteo token into your AQI predictor pipeline without hardcoding it into the repo.

`\${}` — expressions & contexts

The double-curly-brace syntax is how you read dynamic values at runtime — think of it as GitHub Actions' version of variable interpolation. "Contexts" are the built-in objects you can pull data from.

```
{{ github.ref }}           # which branch/tag triggered this run
{{ github.event_name }}    # what kind of event triggered it, e.g. "push"
{{ github.actor }}         # the username who triggered the workflow
{{ matrix.python-version }} # current value inside a matrix build loop
{{ secrets.NAME }}         # a secret's value
{{ env.NAME }}             # an environment variable's value
{{ success() }}            # true if every previous step passed
{{ failure() }}            # true if any previous step failed
{{ always() }}             # evaluates true regardless of outcome
```

`success()`, `failure()`, and `always()` are functions, not variables — you use them inside an `if:` condition to control whether a step runs based on how earlier steps went (e.g. always upload logs, even if tests failed, using `if: always()`).

strategy: matrix

A matrix runs the same job multiple times in parallel, once for every combination of the values you list. This replaces having to copy-paste a job block for each Python version, OS, etc.

```
strategy:
  matrix:
    python-version: ['3.9', '3.10', '3.11']
    os: [ubuntu-latest, windows-latest]
runs-on: ${ matrix.os }
steps:
  - uses: actions/setup-python@v5
    with:
      python-version: ${ matrix.python-version }
```

With 3 Python versions × 2 OSes above, GitHub spins up **6 parallel jobs** automatically — one per combination. Useful for confirming your code works across environments before you commit to one for deployment.

artifacts

A job's filesystem is destroyed the moment it finishes — nothing persists by default. Artifacts are how you save files (test reports, build outputs, model files) so you can download them later from the Actions tab, or pass them to a different job.

```
- uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: coverage.xml

# in a later job:
- uses: actions/download-artifact@v4
  with:
    name: coverage-report
```

Note: artifacts pass data *between jobs* in the same workflow run. Jobs don't share memory or disk directly since each gets its own fresh VM.

if: — conditionals

Placed on a job or a step, if: decides whether that job/step runs at all. If the expression evaluates false, GitHub Actions skips it entirely (shown as grey/skipped in the UI, not a failure).

```
if: github.event_name == 'push'
if: github.ref == 'refs/heads/main'
if: contains(github.event.head_commit.message, '[skip ci]') == false
if: always()           # run this step even if earlier steps failed
if: failure()          # run this step only if something already failed
```

A common pattern: run a "notify on failure" or "upload logs" step with if: failure(), so it only fires when something actually broke.

permissions

By default, the auto-generated GITHUB_TOKEN used inside a workflow has broad-ish permissions. The permissions block lets you explicitly restrict (or grant) what the workflow is allowed to do — good security practice, especially for workflows that write back to the repo (like auto-labeling issues, which the video description mentions as an example use case).

```
permissions:
  contents: read      # read repo files
  issues: write       # allowed to label/comment on issues
  pull-requests: write # allowed to comment/label PRs
```